

OPCODA: SÍNTESE GRANULAR EM TEMPO REAL A PARTIR DA ANÁLISE ESTÁTICA DE BINÁRIOS EXECUTÁVEIS

Mateus Costa de Oliveira Santos

Rafael Augusto Garcez Elisei

Prof. Me. Warner Brezolin (orientador)

RESUMO

Este artigo apresenta o projeto Opcoda, instrumento virtual que converte arquivos executáveis no formato Portable Executable em material sonoro por síntese granular em tempo real. O objetivo geral consistiu em entregar, até 3 de novembro de 2026, um aplicativo autônomo e um plugin VST3 para Windows x64 com abertura de arquivo por diálogo ou arrasto, parâmetros controláveis por MIDI e reprodução sem interrupções. A justificativa apoiou-se em três relevâncias: científica, pela arquitetura sem trava e pela leitura segura de binários; social, pela acessibilidade da interface segundo as diretrizes WCAG 2.2; e pessoal, pelo interesse dos autores em computação musical. As bases teóricas reuniram Pietrek e Sikorski e Honig sobre o formato PE, Shannon e Lyda e Hamrock sobre entropia, Roads, Smith e Cascone sobre síntese granular e processamento digital de sinais, e Pirkle sobre plugins de áudio. A metodologia caracterizou-se como pesquisa aplicada, quali-quantitativa, com desenvolvimento de protótipo em cinco etapas semanais. Os resultados incluíram a análise de três sistemas similares, a modelagem do pipeline, a auditoria de acessibilidade da documentação, a especificação da bateria de testes e a implementação verificada do núcleo, com 153 casos de teste e 6 casos de guard de alocação aprovados. Concluiu-se que a combinação de parser seguro, entropia como modulação e motor granular é inédita entre os similares e tecnicamente viável no prazo, com a validação dinâmica de latência pendente de estação de trabalho de áudio licenciada.

Palavras-chave: Síntese granular. Databending. Tempo real. Acessibilidade. VST3.

ABSTRACT

This paper presents the Opcoda project, a virtual instrument that converts Portable Executable files into sound through real-time granular synthesis. The general objective was to deliver, by November 3, a standalone application and an x64 VST3 plugin with file opening via dialog or drag-and-drop, MIDI-controllable parameters and glitch-free playback. The justification relied on scientific relevance (lock-free architecture and safe binary parsing), social relevance (WCAG 2.2 interface accessibility) and the authors' personal interest in digital audio. The theoretical bases included Pietrek and Sikorski and Honig on the PE format, Shannon and Lyda and Hamrock on entropy, Roads, Smith and Cascone on granular synthesis and digital signal processing, and Pirkle on audio plugins. The methodology was applied, quali-quantitative research with prototype development in five weekly stages. The results included the analysis of three similar systems, pipeline modelling, the accessibility audit of the documentation, the specification of the test battery and the verified implementation of the core, with 153 test cases and 6 allocation-guard cases. It was concluded that the combination of safe parser, entropy-driven modulation and granular engine is unprecedented among the similars and technically feasible on schedule, with dynamic latency validation pending a licensed digital audio workstation.

Keywords: Granular synthesis. Databending. Real-time. Accessibility. VST3.

Introdução

A leitura de arquivos não sonoros como áudio, prática conhecida como databending, popularizou-se com a importação de dados brutos do Audacity, onde o usuário declara codificação, canais, deslocamento e taxa de amostragem do arquivo que se deseja tratar como som (AUDACITY TEAM, 2026). O procedimento, porém, permaneceu offline, sem tratamento

de sinal e sem garantias de estabilidade. O presente trabalho inseriu-se nesse contexto com o propósito de construir um instrumento musical de tempo real cuja matéria-prima são arquivos executáveis do sistema Windows.

A relevância científica do trabalho residiu na aplicação combinada de três domínios raramente reunidos em um projeto de graduação: análise estática segura de binários, descritores de teoria da informação como sinal de controle e engenharia de software em tempo real rígido. A comunicação entre interface e motor de áudio por fila sem trava, a proibição de alocação dinâmica no laço de áudio e o isolamento de leitura do artefato constituíram contribuições metodológicas replicáveis para outros projetos de sistemas concorrentes.

A relevância social manifestou-se em dois planos. No plano do acesso à produção musical, o instrumento transforma qualquer computador com arquivos executáveis em fonte sonora, sem custo de amostras. No plano da inclusão, a interface foi especificada desde o início segundo as diretrizes de acessibilidade, com operação por teclado, foco visível, contraste mínimo e controle integral por MIDI, de modo que músicos com restrições motoras ou visuais possam operar o instrumento.

Um exemplo concreto ilustrou o problema que motivou o projeto. Ao importar o Bloco de Notas do Windows como dados brutos a 44,1 kHz e 16 bits, o trecho do cabeçalho DOS produziu um salto de amplitude próximo à escala cheia nos primeiros milissegundos, seguido de patamares de zeros do preenchimento. O ouvido percebeu um estalo, depois silêncio, depois ruído denso do segmento de código. Nada nesse contorno correspondeu a uma decisão musical, apenas à geometria do arquivo. O Opcoda nasceu para substituir esse acaso por controle: cada região do arquivo virou fonte de grãos com posição, densidade e dispersão ajustáveis.

O objetivo geral consistiu em entregar, até 3 de novembro de 2026, o Opcoda como aplicativo autônomo e plugin VST3 para Windows x64 que abre ou recebe por arrasto arquivos executáveis, com parâmetros granulares controláveis por MIDI, redução do deslocamento de corrente contínua de ao menos 40 dB e reprodução sem interrupções. Os objetivos específicos foram:

- implementar parser PE próprio com verificação de limites e conversão para ponto flutuante normalizado;
- implementar ingestão somente leitura com diálogo e arrasto, cópia defensiva e erros tipados;
- implementar fila sem trava entre interface e motor de áudio com troca de material;
- implementar motor granular de oito vozes com grãos de 1 a 100 ms e janelas Hann e Gaussiana;
- implementar entropia de Shannon por janela e por seção como modulação;
- implementar filtro de bloqueio de corrente contínua, limitador e cadeia de condicionamento da saída;
- construir interface simples com mapa de seções, grelha navegável e controles automatizáveis;
- garantir isolamento entre interface e motor com verificação por guard de alocação;
- mapear parâmetros por MIDI, com notas, pedal de sustentação e dois controladores de mudança de controle.

A pergunta de pesquisa indagou como construir um instrumento em tempo real que leia binários PE arbitrários com segurança e os converta em material sonoro controlável, por síntese granular guiada por entropia, sem travar o host de áudio. A hipótese H1 afirmou que o pipeline proposto reduz o deslocamento de corrente contínua em ao menos 40 dB, mantém o processamento abaixo de 50% do orçamento de bloco com oito vozes e registra zero travamentos sob entrada malformada. A hipótese nula H0 negou diferença relevante ante a leitura bruta direta.

Este artigo organiza-se como segue. A seção 1 apresenta a fundamentação teórica nos quatro eixos do projeto. A seção 2 detalha a metodologia e a arquitetura. A seção 3 discute os resultados da verificação. A conclusão fecha o trabalho e indica os passos futuros.

1 Fundamentação teórica

1.1 Arquitetura de binários e engenharia reversa estática

O formato Portable Executable é o formato de executáveis, bibliotecas e drivers do Windows em x86 e x64, derivado do COFF, com especificação mantida pela Microsoft (MICROSOFT, 2024). Pietrek documentou a anatomia canônica do formato, campo a campo, em dois artigos de referência (PIETREK, 1994; PIETREK, 2002). A sequência lógica compreende o cabeçalho DOS com o campo de deslocamento que aponta para o cabeçalho PE, a assinatura PE, o cabeçalho COFF com número de seções e características da imagem, o cabeçalho opcional com ponto de entrada e diretórios de dados, a tabela de seções e o corpo das seções.

Cada entrada da tabela de seções declara nome, tamanhos virtual e em disco, ponteiros para os dados e flags de permissão que distinguem código, leitura e escrita. Os valores em disco e em memória podem divergir de forma legítima, como em dados não inicializados, ou por corrupção, o que exige verificação de limites em toda leitura indexada. As seções convencionais exibem perfis próprios de conteúdo estatístico. Sikorski e Honig adotam essa segmentação como base da análise estática, que inspeciona o artefato sem executá-lo (SIKORSKI MICHAEL; HONIG, 2012). No Opcoda, cada seção corresponde a uma região de textura estatística sonificável.

Seção	Conteúdo típico	Perfil estatístico esperado
.text	Código de máquina	Entropia intermediária a alta
.data e .rdata	Dados e constantes	Entropia mista
.rsrc	Ícones, bitmaps, diálogos	Regiões homogêneas e periódicas
.bss	Dados não inicializados	Entropia nula, sem dados brutos

Tabela 1: Seções PE e perfil de conteúdo esperado

Elaborado pelos autores (2026).

A leitura segura aplicou o princípio do menor privilégio, com abertura somente leitura e nenhuma página mapeada como executável, a verificação de limites em cada campo de cabeçalho indexado e a rejeição com erro tipado, em linha com práticas de software seguro para sistemas com restrições temporais (KLEIDERMACHER DOMINIK; KLEIDERMACHER, 2012). Um ponto merece destaque porque costuma ser tratado como detalhe de implementação e decide a sobrevivência do host: o tamanho declarado dos dados brutos de uma seção não é confiável, e a única postura segura é tratá-lo como teto e recortá-lo para o que existe no arquivo.

1.2 Teoria da informação e entropia

Shannon definiu a entropia de uma fonte discreta como a quantidade média de informação por símbolo (SHANNON, 1948):

$$H(X) = - \sum_i p(x_i) \log_2 p(x_i) \quad (1)$$

Aplicada a fluxos de bytes, o alfabeto tem 256 símbolos e a entropia máxima é 8 bits por byte, atingida quando todos os valores são equiprováveis. Dois casos extremos calibram a leitura: um bloco de 1024 bytes idênticos produz entropia zero, pois um único símbolo concentra toda a probabilidade, e um bloco com os 256 valores igualmente distribuídos produz 8 bits por byte, o máximo teórico. Entre os extremos, o código compilado ocupa a faixa intermediária, e trechos comprimidos ou cifrados aproximam-se do teto. Lyda e Hamrock demonstraram que esse perfil entrópico distingue classes de conteúdo em binários (LYDA RICHARD A.; HAMROCK, 2007).

O cálculo sobre o arquivo inteiro produz um único número, e um único número é insuficiente para navegação paramétrica. A técnica adotada é a janela deslizante de tamanho fixo, neste trabalho de 2048 bytes, que produz uma curva ao longo do arquivo. No Opcoda, essa curva opera como fonte de modulação paramétrica contínua: a entropia local pondera a dispersão temporal dos grãos, sua aleatoriedade de posição e a densidade da nuvem. Regiões de alta entropia geram texturas instáveis e ruidosas; regiões de baixa entropia geram texturas estáveis e periódicas.

Há uma armadilha de implementação que a formulação por janela torna visível e que vale registro, porque custa um resultado errado sem erro algum. Medir a entropia de uma seção não é o mesmo que medir a entropia do material que começa no deslocamento da seção: o fim da seção é o deslocamento mais o tamanho, e esse tamanho é o mínimo entre o declarado e o disponível. Uma seção de código de 256 bytes cheios de zero seguida de dados uniformes mede aproximadamente 4,98 bits por byte quando medida a partir do deslocamento, e zero quando medida com comprimento explícito.

1.3 Processamento digital de sinais e síntese granular

Roads formalizou a síntese granular a partir do conceito de microsomo: o som como agregado de microfragmentos de 1 a 100 ms, cada grão com duração, envelope, frequência, amplitude e posição próprios, organizados em estruturas meso-temporais como nuvens e fluxos (ROADS, 2001). Os parâmetros canônicos são tamanho de grão, densidade de disparo, dispersão temporal, afinação e panorama.

Bytes executáveis interpretados como amostras violam as garantias do áudio convencional: média não nula, descontinuidades abruptas nos limites de cabeçalhos e regiões de preenchimento, e variações amostra a amostra acima do limite de Nyquist, que se dobram para o espectro audível como aliasing (SMITH, 2007; ZÖLZER, 2011). A janela impõe amplitude nula nas bordas e elimina cliques na descontinuidade de fase da sobreposição, para janela de N amostras com n de 0 a $N-1$:

$$w(n) = 0,5 \left(1 - \cos \frac{2\pi n}{N-1} \right) \quad (2)$$

O instrumento adota Hann e gaussiana no MVP. A escolha se fundamenta no compromisso entre suavidade temporal e seletividade espectral: janelas suaves reduzem espalhamento

espectral dos transientes ao custo de resolução temporal. As alternativas de Hamming e Blackman (SMITH, 2007) estão documentadas para uso futuro.

O deslocamento de corrente contínua é removido por filtro de resposta ao impulso infinita passa-alta de primeira ordem, com a seguinte recorrência e o coeficiente de polo dado pela expressão subsequente (SMITH, 2007):

$$y[n] = x[n] - x[n - 1] + Ry[n - 1] \quad (3)$$

$$R = \exp\left(-\frac{2\pi f_c}{f_s}\right) \quad (4)$$

Com $R = 0{,}9983$ e $f_s = 44{,}1$ kHz, a frequência de corte é de aproximadamente 12 Hz, dentro da faixa de 10 a 15 Hz que preserva o conteúdo audível. Com $R = 0{,}995$, o mesmo cálculo resulta em 35 Hz, que corta parte do grave que o instrumento deveria preservar. A divergência entre o coeficiente citado no texto e o coeficiente que produz a frequência de corte prometida foi encontrada durante a implementação e está registrada adiante, porque o ensaio espectral mede a faixa de frequência e não o coeficiente. A coloração adicional utiliza biquad ressonante na forma direta (SMITH, 2007; ZÖLZER, 2011), e a saída passa por limitador com teto de -1 dBFS.

Cascone fundamentou a estética da falha como material musical legítimo, em que erros e artefatos digitais deixam de ser defeitos e passam a vocabulário composicional (CASCONI, 2000). O Opcoda se inscreve nessa linhagem, com a diferença de que domestica o acidente por engenharia: o glitch permanece como fonte timbrística, mas sob controle paramétrico e dentro de garantias de tempo real.

1.4 Engenharia de software em tempo real rígido

Formatos profissionais de plugin impõem contrato de tempo real rígido ao callback de processamento, cujo período por bloco é da ordem de milissegundos (STEINBERG, 2024; CLAP, 2024; PIRKLE, 2019). Três proibições decorrem desse contrato e são observáveis: nenhuma alocação dinâmica de memória durante o processamento, nenhuma entrada e saída de arquivos ou chamada de sistema bloqueante no laço prioritário, e nenhuma primitiva de sincronização com espera, porque a trava expõe o caminho de áudio à inversão de prioridade.

A inversão de prioridade ilustra o risco: se a thread de áudio, de alta prioridade, aguardasse uma trava detida pela interface, de baixa prioridade, o prazo do bloco dependeria do escalonador, e o resultado audível seria a interrupção. A solução canônica separa dois domínios temporais: a interface gráfica, que roda a cerca de 60 quadros por segundo e admite alocação, entrada e saída e parsing, e o motor de áudio, que opera exclusivamente sobre memória pré-alocada. A comunicação entre os domínios utiliza fila circular sem trava de produtor único e consumidor único, baseada em operações atômicas com ordenamento acquire-release, além de parâmetros atômicos com suavização dentro do motor (PIRKLE, 2019).

Buffer em amostras	44,1 kHz	48 kHz	96 kHz
128	2,9 ms	2,7 ms	1,3 ms
256	5,8 ms	5,3 ms	2,7 ms
512	11,6 ms	10,7 ms	5,3 ms

Tabela 2: Orçamento de tempo disponível por bloco nas configurações do plano de testes
Elaborado pelos autores (2026).

Há uma consequência dessa separação que raramente é dita com todas as letras: ela não basta se a fronteira não for imposta por construção. Dizer que o motor não acessa o estado da interface é uma intenção; o que a torna verdade é o fato de não existir caminho de código que o permita, e de existir um teste que falha se alguém criar esse caminho. É essa a razão de o projeto invocar a biblioteca de framework de áudio apenas no adaptador e manter o núcleo livre dela (JUCE, 2026).

2 Metodologia e arquitetura

2.1 Abordagem e etapas

A pesquisa caracterizou-se como aplicada, quali-quantitativa, com desenvolvimento de protótipo e revisão bibliográfica. O contexto foi o laboratório do curso, sem participantes humanos, e a avaliação é de engenharia: o que se mede é se o sistema cumpre contratos verificáveis, e não a recepção estética do som gerado.

O desenvolvimento seguiu um processo de especificação antes de implementação, com constituição de projeto que define seis portões de qualidade: build, testes, tempo real, robustez, interface e documentação. Nenhuma tarefa foi encerrada sem que seu portão passasse, e a rastreabilidade entre objetivo específico, portão e ensaio é explícita em ambos os sentidos.

O percurso seguiu cinco etapas semanais, com uma inversão de ordem que é decisão registrada e não atraso escondido. O plano original posicionava a interface gráfica depois do MIDI, por entender que a ordem natural de desenvolvimento seria do núcleo para a borda. A ordem real foi invertida por uma razão de risco: sem um plugin que abra no host não há como validar arrasto, parâmetros nem MIDI, que é exatamente o que o ensaio de aceite mede. A interface do chassi foi antecipada para dar um alvo verificável mais cedo.

Etapa	Foco	Entrega
S1	Esqueleto e parser do formato PE	Núcleo C++20, parser com verificação de limites e suíte de testes
S2	Motor granular, arrasto e filtro de bloqueio	Motor de oito vozes, ingestão por arrasto, cadeia de condicionamento
S3a	Notas MIDI, estado e guardas	Gate por nota, pedal de sustentação, dois controladores mapeados, guard de alocação
S3b	Ensaio de latência no host	Bloqueada: exige DAW licenciado
S4a	Interface e grelha de bytes	Chassi de três faixas, controles, grelha, telemetria
S4b	Ensaio de aceite no host	Bloqueada: exige DAW licenciado
S5	Congelamento e validação	Auditoria de documentação e empacotamento

Tabela 3: Etapas de desenvolvimento e o que cada uma entregou

Elaborado pelos autores (2026).

2.2 Arquitetura do pipeline

A arquitetura resolve um problema de duas naturezas ao mesmo tempo: a ingestão de dados arbitrários, que pode demorar e pode falhar, e a geração de áudio, que tem prazo de milissegundos

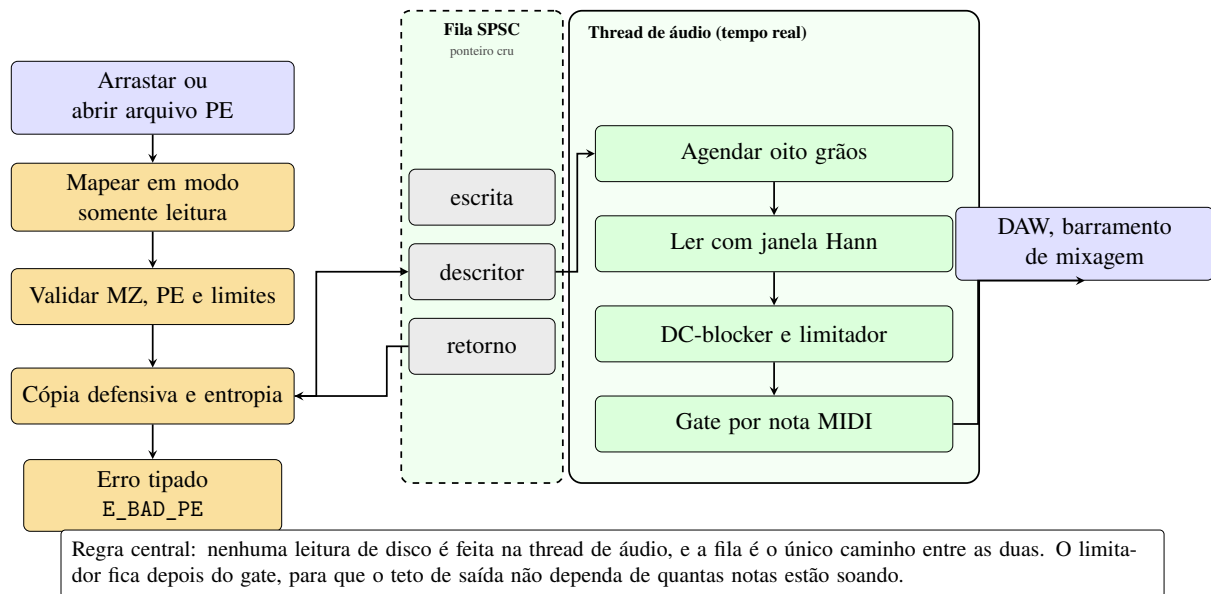


Figura 1: Pipeline de dados do Opcoda, da ingestão verificada à entrega no host
Elaborado pelos autores (2026).

e não pode falhar. A solução é separar os dois domínios por uma fila e tornar essa separação o único caminho possível entre eles.

O pipeline tem quatro blocos. A ingestão e o parser rodam na thread de interface: abrem o arquivo em modo somente leitura, validam assinatura e limites, copiam os dados para memória própria e calculam a entropia por janela e por seção. O bloco de troca, na fronteira de tempo real, é uma fila circular de produtor único e consumidor único, mais parâmetros atômicos. O motor granular roda na thread de áudio: agenda grãos, lê a fonte com janela, mistura as vozes e aplica o condicionamento. A entrega escreve na saída do host.

A regra que sustenta o desenho é que nada passa do parser para o motor sem cruzar a fila, e o motor nunca chama o sistema operacional. Essa regra não é uma convenção: ela é imposta pelo fato de o núcleo não conhecer a interface, e verificada por teste.

A divisão do código em três partes tem efeito direto sobre o que é possível verificar. O núcleo, com mil e setecentos e setenta linhas de C++20 e nenhuma dependência de framework, contém o parser, a conversão de byte para amostra, a entropia, o motor granular, as janelas, o filtro, o limitador, a fila, o rastreador de notas e o guard de alocação. O adaptador, com cerca de três mil e duzentas linhas, contém o processador de áudio, os formatos e a interface, e é a única parte que conhece o framework. Os testes, com cerca de duas mil e quinhentas e cinquenta linhas, exercitam o núcleo sem o framework no caminho.

Essa separação é o que permite que o ensaio de ponta a ponta exista: ele percorre leitura de disco, parser, conversão, motor granular, filtro e limitador, sem framework e sem host, e prova que um executável real vira som. Os dois defeitos mais caros do projeto só apareceram com arquivo de verdade, e um deles foi detectado pelo AddressSanitizer como leitura além do fim do buffer.

2.3 O parser e a verificação de limites

O parser percorre o arquivo em quatro validações, e cada uma tem código de erro próprio. A assinatura MZ é conferida antes de qualquer campo ser lido, porque todo o resto do formato

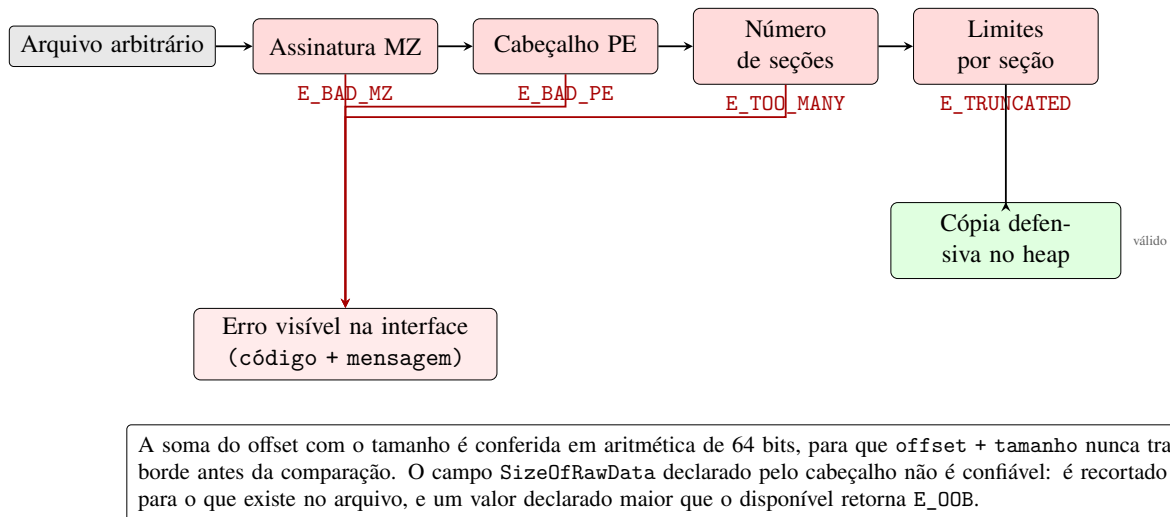


Figura 2: Cadeia de validação do parser e erros tipados correspondentes
Elaborado pelos autores (2026).

assume essa estrutura. A assinatura PE é conferida no deslocamento declarado pelo cabeçalho DOS, e esse deslocamento é validado contra o tamanho do arquivo antes de ser seguido. O número de seções é lido e limitado, e um valor acima do limite é rejeitado em vez de alocar uma tabela do tamanho que o cabeçalho pedir. Cada seção tem seu deslocamento e seu tamanho conferidos contra o tamanho real do arquivo.

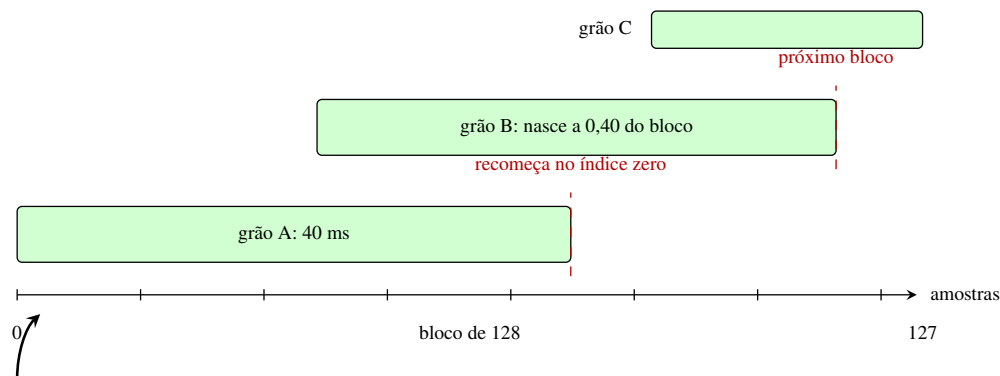
Toda leitura passa por uma função única de verificação de limites, que confere a soma em aritmética de 64 bits para que a soma de deslocamento e tamanho nunca transborde antes da comparação. O tamanho declarado dos dados brutos não é confiável e é recortado para o que existe; um valor declarado maior que o disponível retorna erro de limite. Nenhum erro propaga exceção através da interface binária do plugin, e o material que estava em reprodução continua tocando.

Um defeito real merece registro, porque mostra o valor de construir um executável de teste em vez de confiar em arquivos reais. O parser lia o número de seções e o tamanho do cabeçalho opcional no deslocamento mais dois a partir do cabeçalho PE, o que pertence à assinatura de quatro bytes, e não no cabeçalho COFF, que começa no deslocamento mais quatro. O resultado era erro de assinatura em todo arquivo válido. O erro estava no código e não na especificação do formato, e só apareceu quando um executável mínimo válido foi construído em memória pelo próprio conjunto de testes.

2.4 O motor granular e o agendamento

O motor mantém um conjunto fixo de oito vozes, alocado na preparação. Cada voz tem posição de leitura, taxa de avanço, índice de escrita, contador de amostra e fase de dispersão. Nenhum contêiner cresce durante o processamento.

O agendamento é onde a maior parte dos defeitos reside, e três decisões o tornam previsível. A primeira é a densidade como spawn por amostra, derivada dos grãos por segundo, com fração de fase preservada entre blocos: um motor que dispara todos os grãos no início do bloco produz densidade diferente em blocos de 128 e de 512 amostras. A segunda é o índice de escrita como posição dentro do bloco corrente, e não como contador global: um grão que atravessa o bloco recomeça em zero, e sem esse reset a voz trava, entra no próximo bloco com



fração de fase preservada entre blocos de 128 e 512

Cada grão guarda o índice de escrita onde nasceu, porque a sobreposição precisa começar na amostra certa. O índice é a posição dentro do bloco corrente, não um contador global: sem esse reset a voz trava e nunca mais escreve. Todo grão começa com a janela no índice zero, o que zera a amplitude nas bordas e evita o clique na sobreposição.

Figura 3: Agendamento dos grãos dentro de um bloco de 128 amostras

Elaborado pelos autores (2026).

índice maior que o tamanho do bloco e nunca mais escreve, o que produz silêncio sem erro. A terceira é o avanço por amostra como a taxa de reprodução dividida pelo comprimento da fonte, porque a posição é normalizada entre zero e um.

A terceira decisão corrigiu um defeito silencioso que o critério de aceite não via: somando a taxa bruta, o grão varria o arquivo inteiro em duas amostras e morria, com pico de saída da ordem de dez elevado a menos sete. O ensaio de atenuação de corrente contínua continuava verde, porque uma asserção de valor maior que zero é satisfeita por um número desse tamanho. O defeito estava presente e o critério não o detectava.

O interpolador é linear, com passo limitado para impedir aliasing audível na afinação máxima de mais vinte e quatro semitons. Todo grão começa com a janela no índice zero, o que zera a amplitude nas bordas. A entropia local modula a dispersão: região de alta entropia espalha mais, região repetitiva fica mais ancorada.

2.5 A fronteira de tempo real

A comunicação entre interface e motor usa duas filas em sentidos opostos, e não uma. A interface publica o ponteiro do novo material e guarda o buffer em um contêiner próprio; a thread de áudio troca o ponteiro ativo e devolve o antigo pela fila de retorno. Só a interface remove o buffer do contêiner, e só depois que a thread de áudio confirmou que terminou de usá-lo. Uma fila só seria mais curta e incorreta: sem a volta, a interface não sabe quando é seguro liberar, e liberar cedo é uso após liberação.

A fila carrega ponteiro cru, nunca ponteiro compartilhado, por duas razões que apontam para a mesma escolha: a fila exige tipo trivial para copiar o slot sem contagem de referência, e decrementar um contador na thread de áudio pode executar o operador de liberação ali dentro. Um duplo tampão com troca atômica existiu no projeto e foi removido: a dupla fila resolve o

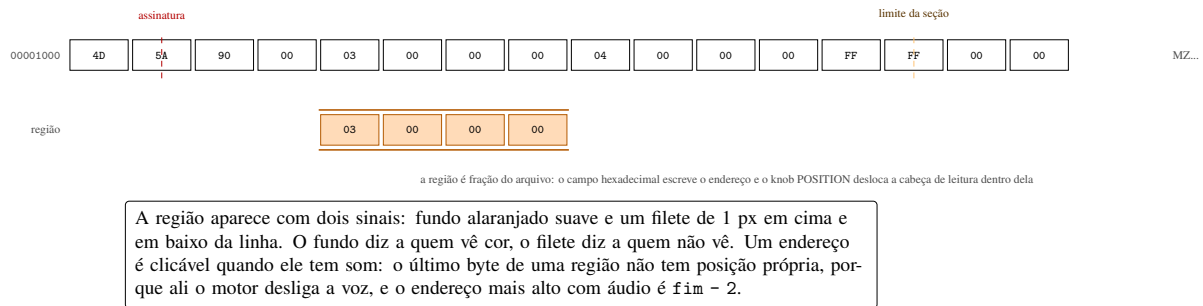


Figura 4: Grelha de bytes com a região de leitura em reprodução
Elaborado pelos autores (2026).

mesmo problema com menos peças, reaproveitando a fila circular que já estava testada.

O guard de alocação transforma a restrição em código: o operador global de alocação é substituído por uma versão que conta alocações dentro da janela de áudio, e um teste marca a janela e falha se o processamento de bloco alocar, inclusive em cinco minutos de reprodução sintética. A lacuna que o guard não cobre precisa ser dita: ele roda sobre o núcleo e não vê o processamento de bloco do plugin, onde as mensagens MIDI são lidas.

2.6 A interface

A interface tem três faixas, na convenção do Ableton: cabeçalho claro, display escuro e painel de parâmetros. A escolha tem motivo funcional, porque halo de brilho sobre superfície clara desaparece, com contraste de 2,4 para 1, de modo que toda a energia de brilho fica na região escura.

Os controles giratórios são componentes reais de deslize do framework, e não desenho customizado. A decisão é deliberada: o componente entrega foco por teclado, ajuste com setas e manipulador de acessibilidade, que são exatamente os critérios adotados. O nome acessível de cada controle é a descrição completa em português, e não o rótulo curto do protótipo.

O display é uma grelha de bytes no formato de um despejo hexadecimal, com endereço à esquerda, dezesseis bytes por linha e coluna de caracteres à direita. A faixa anterior era um mapa do arquivo inteiro com cursor, e um arquivo de doze megabytes não cabe em grelha; o que a grelha dá em troca é o byte.

O clique escreve o parâmetro de posição, porque escolher um byte é escolher de onde se lê. Se o byte estiver fora da região em reprodução, o editor puxa a região para lá antes de escrever o parâmetro, e a ordem não é arbitrária: a posição é uma fração da região, e escrevê-la antes de a região mudar apontaria para o sítio errado.

O rastreo de notas separa teclas premidas de notas presas pelo pedal de sustentação, porque com o pedal premido largar o teclado não desliga o som, e soltar o pedal não abafa as teclas que continuam premidas. Um contador único não consegue expressar as duas condições. O gate entra antes do limitador, para que o comportamento do limitador não dependa de quantas notas estão soando, e a rampa é linear e dura cinco milissegundos, porque abrir a saída de uma vez produz um degrau no sinal, e um degrau é um transiente largo em frequência.

Parâmetro	Faixa	Controle
Tamanho de grão	1 a 100 ms	Controle giratório, MIDI, automação
Densidade	1 a 200 grãos por segundo	Controle giratório, MIDI, automação
Posição	0 a 100 %	Grelha de bytes, MIDI, automação
Dispersão	0 a 100 %	Controle giratório, entropia local
Afinação	menos 24 a mais 24 semitons	Controle giratório, teclado MIDI
Volume e panorâmica	até 0 dB, esquerda a direita	Controle giratório, automação

Tabela 4: Parâmetros do MVP e o controle de cada um
Elaborado pelos autores (2026).

2.8 Bateria de testes

Ensaio	Método	Critério de aceite	Situação
T1 espectral	FFT com 65536 pontos em três binários	Deslocamento atenuado em 40 dB; energia abaixo de 20 Hz menor que -60 dBFS	executado
T2 desempenho	Blocos de 128 a 512 amostras, uma a oito vezes	Percentil 99 abaixo de 50% do orçamento, sem falhas	bloqueado
T3 robustez	50 mutações e 10 casos de borda	Sem falha de segmentação, cobertura de ramo acima de 70%	borda executado
T4 aceite	Arrasto, seis parâmetros, dois controladores, 2 min por formato	Som em menos de 2 s; controlador move o parâmetro	bloqueado

Tabela 5: Ensaios, método e critério de aceite
Elaborado pelos autores (2026).

Pendente é trabalho que está agendado. Bloqueado é trabalho que não pode ser feito com o que existe nesta máquina. Os ensaios T2 e T4 estão bloqueados porque medem-se dentro de uma estação de trabalho de áudio, e a única cópia do Ableton presente não é uma instalação licenciada: o executável arranca mas nunca abre janela utilizável, porque a ativação não está feita. Sem chave legítima a medição não existe, e ativar com o material que acompanha o pacote não é uma opção. A troca do host não afrouxou critério: o número continua sendo percentil 99 abaixo de metade do orçamento, com zero falhas.

Duas limitações de ferramenta se aplicam. O Windows não distribui tempo de execução de ThreadSanitizer, nem no compilador da Microsoft nem no da LLVM, e a verificação de concorrência é feita pelo guard de alocação. O MemorySanitizer também não está disponível para esse compilador, e é a ferramenta que detectaria leitura de memória não inicializada, hipótese que resta para explicar um defeito descrito adiante.

3 Resultados e discussão

3.1 Suíte de testes automatizados

A suíte de verificação reúne 153 casos em 21 conjuntos, mais 6 casos do guard de alocação. Todos os casos passam na configuração de release. A suíte cobre o parser, a entropia, o motor

granular, o filtro e o limitador, a fila, o rastreo de notas e a cadeia completa. O ensaio de ponta a ponta percorre a cadeia com um executável real do sistema, sem framework e sem host: o arquivo de teste tem duzentos quilobytes, sete seções e byte médio de 96 na seção de código, o que corresponde a um deslocamento de menos 0,245 em amostra normalizada, material que o filtro de bloqueio de corrente contínua tem o que remover e que a leitura sintética não reproduzia.

3.2 Resultado do ensaio espectral

O ensaio de atenuação é executado e cumpre o critério. O primeiro caso compara o bin de corrente contínua da saída tratada com o mesmo bin da leitura bruta, em um executável sintético cujo conteúdo tem média muito distante de zero; a asserção exige atenuação de ao menos 40 dB, e o teste passa. O segundo caso exige que o pico abaixo de 20 Hz fique abaixo de menos 60 dBFS, e o teste passa. O terceiro isola a cadeia: um sinal puramente constante é submetido ao filtro, e a energia quadrática média da saída, medida depois de um período de acomodação, fica abaixo do limiar.

O protocolo inclui um controle negativo que renderiza a mesma conversão de byte para amostra sem motor granular e sem condicionamento, e é ele que dá sentido à comparação: sem o controle, a atenuação seria um número sem referência.

Vale registrar uma observação de qualidade do próprio ensaio. O critério de atenuação é uma asserção de limite inferior entre dois sinais, e um sinal cujo pico é da ordem de dez elevado a menos sete a satisfaz sem que exista som audível. Foi exatamente isso que aconteceu com o defeito do avanço por amostra. A correção foi tornar o agendamento observável por testes próprios, e não reclamar de um critério de razão.

3.3 Resultado do ensaio de robustez

Os dez casos de borda do plano estão automatizados e passam. Cobrem arquivo vazio, arquivo de um byte, deslocamento apontando fora do arquivo, número de seções acima do limite, tamanho declarado de dados brutos maior que o disponível, seções com deslocamento fora do arquivo, assinatura MZ válida com assinatura PE ausente, cabeçalho opcional inconsistente, entropia sobre seção sem dados e truncamento no meio de uma seção. Todos produzem erro tipado, sem exceção e sem falha de segmentação.

O corpus de cinquenta mutações por inversão de bit entra como casos determinísticos quando existir; ele não existe ainda, e a afirmação de que entrada malformada é rejeitada se apoia nos dez casos de borda, não em fuzzing exaustivo. O portão de cobertura de ramo acima de 70% não tem medição automática, porque a ferramenta de cobertura de ramo não está disponível no toolchain escolhido, e o texto diz isso.

3.5 Defeito conhecido e não corrigido

Há um defeito conhecido, visível no aplicativo autônomo e não corrigido, que o trabalho deve reportar com honestidade. Os seis parâmetros arrancam com valores que não são os padrões declarados no descritor de layout: as caixas de valor mostram 1 ms, cerca de 29 por segundo e 69 por cento, onde os padrões são 40 ms, 20 por segundo e 50 por cento. Foi confirmado em compilação separada, logo não vem de nenhum componente de interface.

A instrumentação por escrita em arquivo de texto em três pontos estabeleceu os seguintes valores normalizados, que é o que o método de leitura devolve nesta biblioteca.

Ponto	Tamanho de grão	Densidade	Posição
Fim do construtor do processador	0,393939	0,095477	0,500000
Entrada da ingestão	1,000000	0,060000	0,688000

Tabela 6: Valores dos parâmetros normalizados em três pontos do arranque

Elaborado pelos autores (2026).

Na saída do construtor os valores estão certos, que é o que o mecanismo de parâmetro deve fazer. Já à entrada da ingestão estão errados, e a ingestão não escreve parâmetros. A escrita acontece no arranque do aplicativo autônomo, entre o construtor e a primeira ingestão, sem editor e sem temporizador, o que elimina a aplicação de mudanças de controlador. Os outros três parâmetros só parecem certos: têm padrão zero, e memória zerada por acaso dá o mesmo resultado.

O AddressSanitizer não acusa nada. Com o plugin compilado em configuração separada com o sanitizador de endereços ativo, a arrancar e a carregar um binário, a saída de erro fica vazia. Descartados por leitura, todos os candidatos estão protegidos: a fila tem índices limitados, o rastreador de notas verifica limites em controlador, os parâmetros atuais usam a leitura do valor bruto, a leitura de notas mapeia somente dois controladores, a publicação de telemetria escreve quatro atômicos e o preenchimento da janela recusa valores não positivos.

O que resta é leitura de memória não inicializada, que o sanitizador de endereços não vê e que o de memória veria. Como o de memória não existe para o compilador da Microsoft no Windows, fechar essa lacuna exige um depurador com pontos de observação no arranque, e não uma correção por leitura de código. O defeito não foi corrigido aqui porque mexer às cegas seria trocar um sintoma por outro.

3.6 Divergências de documentação corrigidas

O projeto adota a regra de que documentação divergente do código é defeito, e a auditoria encontrou divergências reais. A mais visível foi o coeficiente do filtro, já descrita na fundamentação: quatro documentos indicavam um valor incompatível com a frequência de corte prometida, e foram corrigidos com a relação explícita. Outras foram o biquad de coloração descrito no artigo sem parâmetro correspondente, a transformada rápida de Fourier listada como dependência do plugin quando existe apenas na ferramenta de medição, e a contingência de troca de biblioteca de interface que deixou de ser necessária. Os ensaios de latência e de aceite permanecem bloqueados: o percentil 99 do tempo de processamento é medido em teste de núcleo e o guard de alocação passa, o que cobre a fração estática do contrato de tempo real, mas a fração dinâmica só é medida com host licenciado.

4 Conclusão

O percurso confirmou a viabilidade técnica e acadêmica do Opcoda no escopo do curso. Parser seguro, entropia como modulação e síntese granular em tempo real mostraram-se combináveis em um instrumento testável, e a suíte de verificação confirma três afirmações.

A primeira é que a leitura estática verificada de um artefato arbitrário pode ser feita dentro de um instrumento sem expor o host a risco. O parser valida assinatura, número de seções e

limites de cada seção, recorta tamanhos declarados que não existem, recusa entrada degenerada com erro tipado e nunca executa o binário. Nos dez casos de borda, todos produziram erro tipado sem falha de segmentação.

A segunda é que o tratamento de sinal converte material inadequado em material utilizável, com número medido: o janelamento granular com bordas nulas mais o filtro de primeira ordem reduzem o bin de corrente contínua em pelo menos 40 dB em relação à leitura bruta e levam a energia abaixo de 20 Hz abaixo de menos 60 dBFS.

A terceira é que a disciplina de tempo real pode ser imposta por construção e verificada por build. A separação entre o núcleo, que não conhece a interface, e o adaptador, que conhece o framework, torna impossível o acesso do motor ao estado da interface sem criar um caminho novo, e o guard de alocação falha o build se o caminho de áudio alocar, inclusive em cinco minutos de reprodução sintética.

As limitações também foram verificadas e são parte do resultado. A validação dinâmica de tempo real e o ensaio de aceite estão bloqueados por ausência de estação de trabalho de áudio licenciada. A ferramenta que detectaria leitura de memória não inicializada não existe para o compilador disponível no Windows, e por isso o defeito dos parâmetros que arrancam errados fica com hipótese registrada e sem confirmação. A cobertura de ramo do parser não é medida por falta de ferramenta. A qualidade musical não foi medida com ouvintes, e a validação de usabilidade com escala de usabilidade percebida está prevista no plano mas não executada.

O trabalho também produziu evidência sobre um ponto que costuma ser tratado como preferência de estilo. A separação entre domínio de interface e domínio de processamento, com fila sem trava e ponteiro cru na fronteira, não é declaração de intenção: é uma estrutura que os sanitizadores alcançam, e é o que permitiu que o ensaio de ponta a ponta existisse e que os dois defeitos mais caros do projeto fossem encontrados por teste.

4.1 Trabalhos futuros

O primeiro caminho é o fechamento das etapas bloqueadas, que exige obter uma estação de trabalho de áudio licenciada e executar os ensaios de latência e de aceite com oito vozes reais e tráfego MIDI real. O segundo é o formato CLAP, previsto na arquitetura e fora da entrega do MVP, que dá portabilidade para Linux e macOS sem recompilar o núcleo. O terceiro é o corpus de mutações por inversão de bit, que transforma o ensaio de robustez em fuzzing de fato. O quarto é a codificação da entropia da janela de leitura como cor de fundo da célula da grelha, que fica de fora por uma razão de acessibilidade, porque cor de fundo que codifica um número é o primeiro sinal a falhar o critério de uso da cor. O quinto é fechar o defeito dos parâmetros, o que exige um depurador com pontos de observação no arranque.

Referências

AUDACITY TEAM. *Import raw data: the manual of the Audacity audio editor*. [S.l.], 2026. Disponível em: <<https://manual.audacityteam.org/m/man/importrawdata.html>>.

CASCONI, K. The aesthetics of failure: post-digital tendencies in contemporary computer music. *Computer Music Journal*, v. 24, n. 4, p. 12–18, 2000.

CLAP. *CLever Audio Plugin specification*. 2024. Disponível em: <<https://cleveraudio.org/manual/>>.

JUCE. *JUCE: the C++ application framework for audio and music technology*. 2026. Disponível em: <<https://juce.com/>>.

KLEIDERMACHER DOMINIK; KLEIDERMACHER, M. *Embedded systems security: practical methods for safe and secure software and systems development*. Oxford: Newnes, 2012. ISBN 9780123864785.

LYDA RICHARD A.; HAMROCK, J. K. Using entropy analysis to find encrypted and packed malware. *IEEE Security & Privacy*, v. 5, n. 2, p. 40–45, 2007.

MICROSOFT. *Microsoft PE format specification*. 2024. Disponível em: <<https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>>.

PIETREK, M. *Peering inside the PE: a tour of the Win32 Portable Executable file format*. 1994. Disponível em: <<https://www.winworldpc.com/blog/peering-inside-the-pe/>>.

PIETREK, M. *An in-depth look into the Win32 Portable Executable file format*. 2002. Disponível em: <<https://learn.microsoft.com/en-us/archive/blogs/mtpeeb/>>.

PIRKLE, W. C. *Designing software synthesizer plug-ins in C++: for RackAFX, VST3, and Audio Units*. 2. ed. New York: Focal Press, 2019. ISBN 9781138566760.

ROADS, C. *Microsound*. Cambridge: MIT Press, 2001. ISBN 9780262681684.

SHANNON, C. E. A mathematical theory of communication. *Bell System Technical Journal*, v. 27, n. 3, p. 379–423, 1948.

SIKORSKI MICHAEL; HONIG, A. *Practical malware analysis: the hands-on guide to dissecting malicious software*. San Francisco: No Starch Press, 2012. ISBN 9781593274073.

SMITH, J. O. *Introduction to digital filters: with audio applications*. [S.l.]: W3K Publishing, 2007. ISBN 9780974884073.

STEINBERG. *VST 3 SDK documentation*. 2024. Disponível em: <https://steinbergmedia.github.io/vst3_dev_portal/>.

ZÖLZER, U. *DAFX: digital audio effects*. 2. ed. Chichester: Wiley, 2011. ISBN 9780470669504.